

ФСР · карьера

Техническая документация и отчёт проверки MVP 1.0.0

Обратный найм по подтверждённым навыкам
7 октября 2026 · UTC

Код: github.com/ImmortalDR/CV_hakaton
Рабочая ветка: `codex/fsp-mvp`

Демонстрационная реализация. Результаты синтетических измерений не являются экспертной валидацией профессиональных грейдов.

Архитектура MVP 1.0.0

Один React/TypeScript frontend, один FastAPI backend и одна PostgreSQL 16. Nginx раздаёт статические файлы и проксирует /api. GPU, платные модели, внешние вакансии и сторонний архив для запуска не нужны.

Кандидат / работодатель → React + Nginx → FastAPI → PostgreSQL

FastAPI: банк тестов, оценка, подбор, PDF, демонстрационный провайдер ФСП.

apps/api/fsp/models.py содержит 14 таблиц. Пользователь отделён от компании через CompanyMember. В MVP один пользователь принадлежит одной компании; управление сотрудниками через UI отложено. Категории, грейды, навыки и семейства задач — версионные справочники bank.py, без отдельного CRUD администратора.

User хранит роль, хеш пароля и подтверждение e-mail. Session и Verification хранят только SHA-256 непрозрачных случайных токенов и срок действия. Profile разделяет claimed_grade и verified_grade, хранит ссылку на успешную попытку, настройки видимости, самодекларацию и достижения. Attempt хранит seed, версию, задания, эталоны, ответы и результат; SkillEvidence связывает конкретный навык с попыткой, долей верных ответов и датой. Result находится в JSON попытки, отдельно продублированы структурированные доказательства.

Need хранит подтверждённые структурированные требования. Snapshot хранит порядок кандидатов, методику, основания и обезличенные доказательства на дату поиска. При чтении снимка повторно проверяются публикация, скрытие стажа и адресное право на контакты; имя и контакты не кэшируются в снимке. Грейд/доказательства в снимке исторические, актуальный PDF строится по текущему профилю. Редактирование запроса создаёт новый снимок, старый не перезаписывается.

Invitation содержит условия, компанию и контакт работодателя на момент отправки. ContactGrant — отдельная запись (candidate_id, company_id). Принятие и выдача права происходят в одной транзакции. Message принадлежит приглашению. Consent хранит историю согласий с версией текста и датой.

Подбор

Сначала точная специализация и один из выбранных подтверждённых грейдов; неподтверждённый профиль есть только в отдельной группе банка. Затем внутри каждой категории применяется matching-1.0.0:

$60 \times \text{доля met среди обязательных} + 15 \times \text{доля met среди желательных} + 0,2 \times \text{результат успешного теста} + \text{бонус ФСП (0...5)}$.

Пустой список требований даёт нулевую часть, без неявного перераспределения весов. Индекс — не вероятность и не процент пригодности. Достижения демопровайдера дают 1/2/3 балла, общий вклад ограничен 5. Текст, ФИО, контакты, пол, возраст не участвуют. Сортировка между грейдами лишь группирует карточки; высокий спортивный бонус не переводит человека в другую категорию.

met — есть положительное доказательство выбранного успешного теста; unmet — критерий проверен, порог навыка не достигнут; unknown — проверки нет. Даже числовой стаж остаётся самодекларацией, поэтому требование стажа получает unknown и не добавляет баллы. Ноль лет и неизвестный стаж различаются.

Описание потребности сохраняется для контекста. Пользователь сам задаёт структуру и подтверждает её; автоматического NLP-разбора нет. Обязательный навык — критерий оценки, а не жёсткое исключение: слабые совпадения остаются видимыми с unknown/unmet. Контрольный запрос React в оценке показывает ограничение этого подхода. В дальнейшем можно добавить явный режим «только все обязательные навыки подтверждены».

Транзакции и безопасность

Блокировка строки Profile сериализует старт/сдачу тестов, принятие приглашения и отзыв согласия. Company блокируется при создании приглашений; ключ запроса уникален внутри компании. Повтор одного payload возвращает ту же запись, изменение payload с тем же ключом даёт 409. Принятое/отклонённое решение терминально; повтор того же статуса безопасен, противоположное решение — 409.

Транзакция БД завершается до отправки ответа: Depends(..., scope="function"). Это исключает ответ об успешном создании сессии до фиксации записи. Семантика scope сверена с документацией FastAPI (<https://fastapi.tiangolo.com/tutorial/dependencies/dependencies-with-yield/#early-exit-and-scope>).

Доступ проверяется в API по роли, владельцу и компании. Один и тот же projection() используется для выдачи и PDF. До согласия скрыт весь произвольный текст кандидата, включая контакты в ФИО/описании/ролях. PDF создаётся для текущего получателя, в памяти, без публичного файла. Ответы API имеют Cache-Control: no-store. React экранирует текст, PDF экранирует XML. Произвольный код кандидата не исполняется.

Отзыв публикации/обработки удаляет ContactGrant. Исторические приглашения остаются у сторон, но без контактов; диалог закрывается. Повторное включение публикации и повтор запроса принятия старого приглашения право не возвращают. Нужно новое приглашение и новое принятие. Уже скачанный третьей стороной PDF технически отозвать невозможно.

Пароли — Argon2id. Cookie — HttpOnly, SameSite=Lax, 24 часа; Secure включается для HTTPS через окружение. Подтверждение почты — один час, один раз. Изменяющие запросы требуют X-Requested-With: fsp-web; CORS не включён, nginx ограничивает тело 64 KiB. Auth имеет ограничение 20 запросов в минуту на адрес процесса; это простой локальный limiter, не распределённый сервис.

Банк и рубрика

Версия банка 1.1.0, два направления и три уровня. 12 собственных семейств, в каждой попытке два варианта каждого из двух семейств категории: четыре числовых ответа за 30 минут. Один верный ответ — 25 баллов, подтверждение выбранного грейда — от 75. Навык считается met при не менее 75% ответов своего семейства; при двух вопросах это 2/2. Успешный тест с 3/4 поэтому может оставить один навык unmet. Это видно работодателю отдельно от общего грейда.

Категория	Семейства	Что проверяется
Python Junior	py_filter, py_boundary	фильтрация/сумма; ошибки включения границ
Python Middle	py_alias, http_keys	общие ссылки вложенных списков; идемпотентность
Python Senior	py_dag, http_backoff	зависимости параллельных задач; время повторов API
Data Junior	sql_where, stats_median	фильтр/агрегация; медиана
Data Middle	sql_join, stats_weighted	отсутствие связей при LEFT JOIN; взвешенная конверсия
Data Senior	sql_rank, tx_snapshot	плотный ранг; потерянное обновление

Параметры: $a=2\dots9$, $b=2\dots8$; основной список — семь чисел $1\dots20$. Граничные входы выбираются из $a-2\dots a+b+2$ с возможными повторами. Для DAG $c=2\dots12$; повторы HTTP — 3...5 попыток; восемь идемпотентных запросов из четырёх ключей; шесть заказов с customer_id из 1...4 при пяти клиентах. Точные генераторы, эталонные формулы и объяснения находятся в `apps/api/fsp/bank.py`.

Используется отдельный `random.Random(seed)`, версия и полный вариант сохраняются в PostgreSQL. SHA-256 текста набора уникален в БД; генератор пробует до 20 новых seed при совпадении. Клиент не выбирает seed и не присылает оценку. До завершения в API только текст и навык. После завершения доступны правильные ответы и объяснения; повторная сдача сохраняет первый результат.

Ответ обрабатывается Decimal; поддерживаются точка/запятая, пробелы вокруг числа. Допуск 0,005 для округления. NaN, Infinity, пустые и нечисловые ответы не засчитываются. Генерация не использует LLM в runtime и не запускает пользовательский код. Набор `evaluation/oracles.py` отдельно решает задачи: SQL выполняется в памяти, Python-семантика и расписания моделируются пошагово.

История и изменения уровня

Первый неуспешный тест не присваивает грейд. Можно сразу самостоятельно выбрать другой, в том числе более низкий уровень. Пересдача той же пары специализация/уровень — через 24 часа от старта последней попытки. После подтверждения смена грейда, включая добровольное понижение, возможна через 90 дней от последней успешной смены. Тест того же уровня разрешён с отдельным интервалом 24 часа. Границы включённые: ровно 24 часа или 90 дней — разрешено. Повторное успешное подтверждение того же грейда не сдвигает дату смены. Провал и истечение времени сохраняют прежний грейд.

В один момент у пользователя одна активная попытка. При смене вкладки она восстанавливается. До завершения попытки специализацию менять нельзя; после первого подтверждения специализация в MVP закреплена. Переход на другую специализацию после подтверждения пока не поддерживается.

Границы выводов

Числовая параметризация затрудняет распространение конкретных ответов, но решение семейства остаётся узнаваемым. Нет прокторинга и защиты от LLM, подсказок другого человека или выученного алгоритма. Четыре задания не покрывают все обязанности Junior/Middle/Senior. Названия уровней — рубрика демонстрации, а не подтверждённый профессиональным сообществом стандарт.

Версия 1.1.0 исправляет постоянный ответ граничного семейства из черновой 1.0.0. Сохранённые ранние демопопытки остаются со своей версией и своим эталоном. Они не перезаписываются при обновлении банка.

Фактические числа, матрица ошибок и ограничения оценки приведены в `VALIDATION.md` и `evaluation/results.json`. Следующий необходимый шаг — слепая оценка заданий профильными экспертами и пилот на людях с несколькими вариантами, независимыми метками, анализом сложности и пограничных результатов.

Фактические результаты проверки

Проверка проведена 7 октября 2026, UTC, на Linux x86_64, 2 логических CPU, около 3,82 GiB RAM. Исходные логи локально в `audit/current/`; публикуемые машиночитаемые результаты — рядом с этим документом и в `evaluation/`. Ниже разделены исполненные проверки и невыполненная внешняя валидация.

Исполненные проверки

Проверка	Команда / артефакт	Результат
Backend, правила, права, конкурентность	<code>python3 scripts/run_local.py</code> <code>.venv/bin/pytest -q tests</code>	pass: 36 тестов; 2 предупреждения устаревания в инструментах тестирования
TypeScript + production build	<code>cd apps/web && npm run build</code>	pass, 53 модуля, JS около 240 kB / 73 kB gzip
Образы и запуск	<code>docker compose build, docker compose up -d --wait</code>	pass, отдельный проект fsp-mvp, PostgreSQL/API доступны
Новая БД, миграция, повтор seed	<code>scripts/check_install.py</code>	pass: 14 таблиц, 15 пользователей после первого и второго seed, schema drift отсутствует
Сохранение данных при перезапуске	<code>scripts/check_restart.py,</code> <code>restart-results.json</code>	результат и хеш строк зафиксированы в JSON
Chromium, desktop + mobile	<code>tests/browser_flow.py,</code> <code>browser-result.json</code>	сквозной сценарий без моков; отдельные синтетические регистрации
API-контракт	openapi.json из запущенного сервера	OpenAPI 3.1, 27 операций, 35 моделей, cookie/CSRF/ошибки/PDF
Зависимости frontend	<code>npm audit --json</code>	pass: известных уязвимостей на момент проверки не найдено
Зависимости backend	<code>pip-audit -r apps/api/requirements.lock</code> <code>--no-deps --disable-pip</code>	pass: известных уязвимостей на момент проверки не найдено
Нагрузка	<code>tests/load_probe.py,</code> <code>load-results.json</code>	pass: 270 запросов, 30 разных сессий/пользователей, 0 ошибок

Интеграционные проверки выполнялись на настоящем PostgreSQL, в отдельной БД `fsp_test`. Проверены одноразовые/истёкшие токены, Argon2 и хешированные сессии, роли, подмена `seed/score/grade`, отсутствие эталона до сдачи, история, границы 24 часов и 90 дней, добровольное понижение и сохранение грейда при провале/истечении времени. Параллельные старты, сдачи, приглашения и принятия дают по одному результату и праву доступа.

Проверены запреты чужому кандидату и второй компании: API, поиск, снимок, PDF с кириллицей, приглашения и чат. Контакты намеренно размещены в имени, телефоне и произвольных полях тестового профиля. До принятия они скрыты. Отзыв публикации закрывает доступ, скрытие стажа редактирует старые объяснения, повтор принятия после повторной публикации не возвращает старое разрешение. Нулевая/отрицательная/перевернутая вилка, дробные рубли, чужая валюта и пустое описание отвергаются. Подтверждение ФСП не меняет грейд.

Нагрузочный замер

30 разных синтетических работодателей, по 9 запросов каждый, одновременно не более одного запроса от пользователя. 13 опубликованных профилей. Три круга чтения банка, карточки и PDF: всего 270 ответов 200 за 11,16 с, 24,19 запроса/с. Медиана общего времени 706 мс, p95 3040 мс.

Операция	Запросов	Медиана, мс	p95, мс
Банк	90	2119,86	3196,42
Карточка	90	514,20	796,52
PDF	90	630,00	866,41

Сессии для этого замера созданы как явные тестовые данные в БД; регистрация и почта не включены в нагрузку. Нет измерения долгого потока, записи или большого корпуса. Задержка банка около 3,2 с на p95 — оставшееся ограничение N+1 и полной выдачи, а не доказательство готовности к промышленному масштабу.

Независимые вычисления и синтетические уровни

Протокол зафиксирован до финального измерения. `evaluation/oracles.py` не импортирует генератор или оценщик. Unit-набор проверяет 600 уникальных вариантов, 2400 вопросов по 6 категориям; воспроизводимость и `blueprint`. Отдельная оценка содержит 12 синтетических персон, 20 вариантов для каждой, 3 уровня: 720 уникальных попыток и 2880 проверок эталона независимым решением.

Для персон заранее заданы известные семейства и ожидаемый уровень; у двух «noisy-middle» дополнительно вероятность ошибки 0,2. Получено 235/240 совпадений ожидаемого уровня: 97,92%. Средняя доля наиболее частого результата по вариантам — 97,92%. Это симуляция правила, не доля правильно оценённых реальных программистов.

Ожидаемый → полученный	Не подтверждён	Junior	Middle	Senior
Не подтверждён	80	0	0	0
Junior	0	40	0	0
Middle	2	3	75	0
Senior	0	0	0	40

Пять ошибок — чувствительность короткого теста к случайным пропускам у синтетических Middle. Четыре вопроса не дают надёжной оценки человека по всем компетенциям уровня. Эту слабость нельзя устранить заявлением о «равной сложности» параметрических вариантов; нужен пилот и более широкий банк.

Качество подбора

Train отсутствует: модель не обучается, веса фиксированы. Dev: 16 профилей, 2 запроса. Test: 48 профилей, 8 запросов, другие ID. Это разные вымышленные персоны, но специально повторяемые типы ошибок и категории; статистической независимости выборки реальных людей эти данные не обеспечивают. Явные `relevance labels` хранятся в `fixture`, не получены вызовом `rank_candidate`. Автор разметки — разработка; независимых экспертов ФСП: 0.

Baseline: пересечение заявленных навыков и стаж внутри той же категории. P@5 всегда делит число релевантных в первых пяти на 5, даже при нехватке релевантных кандидатов. Это намеренно сурово к пустой выдаче и коротким спискам.

Запрос test	P@5 MVP	P@5 baseline
Python Junior	1,00	0,60
Python Middle	1,00	0,60
Python Senior	1,00	0,60

Запрос test	P@5 MVP	P@5 baseline
Data Junior	1,00	0,60
Data Middle	1,00	0,60
Data Senior	1,00	0,60
Python + React без доказательства React	0,00	0,00
Python + обязательное тестирование	0,40	0,20
Среднее	0,80	0,475

В шести простых случаях доказательства вытесняют громкие самодекларации. В запросе React все пять выдаваемых карточек нерелевантны по разметке; интерфейс честно показывает unknown, но выдача всё равно заполнена. В запросе с двумя обязательными навыками есть только два полностью релевантных кандидата; оставшиеся три получают неполное соответствие. Эти ошибки сохранены в `evaluation/results.json`. Веса после просмотра test не перенастраивались.

Найденные и исправленные проблемы

- Черновое семейство граничных условий всегда имело ответ 2. В 1.1.0 набор входов меняется содержательно; независимый `oracle` проверяет расхождения.
- Историческое объяснение могло раскрыть скрытый позже стаж: теперь оно проверяет актуальную настройку при каждом чтении.
- Первое выполнение проверки 90 дней вернуло 401: истекла тестовая сессия. Исправлен тест часов с явным продлением своей сессии; правило API не ослаблено.
- Первоначальная frontend-сборка завершилась 137 из-за памяти. Отдельно найден рекурсивный анализ Rollup; настройка сборки исправлена и перепроверена.
- `npm audit` обнаружил проблему Vite 6.4.1; обновлён до 6.4.4. `pip-audit` обнаружил 7 записей у Starlette 0.46.2; FastAPI/Starlette обновлены до 0.142.3/1.3.1, полный набор тестов повторён. Финальные сканы чистые относительно доступных им баз, не являются полным аудитом безопасности.

Сохранность исходных материалов

`python3 scripts/verify_workspace.py --inputs-only`: 8141 исходный файл совпадает с предоставленными SHA-256; 8131 архивный путь разрешён через фактический вложенный каталог `fsp_mvp_bundle`. 22 редактируемых рабочих файла исключены из этой проверки. Первоначальный запуск без учёта вложенности дал ложные missing и ожидаемые отличия рабочих документов; это исправление пути проверки, а не восстановление или перезапись исходников. Хеширование не объявляется содержательным чтением.

Не запускалось / не подтверждено

Реальная доставка SMTP, внешний ФСП/Keycloak, публичный HTTPS-домен, экспертная разметка и пилот с людьми, прокторинг и защищённый промышленный режим обработки персональных данных. Нет утверждения о завершённом чтении всех файлов исходной подборки: реальные статусы сохранены в `docs/review`.

Итоговая матрица соответствия

Исходные формулировки и источники сохранены в docs/REQUIREMENTS_MATRIX.md. pass означает исполненный сценарий в указанной границе MVP, не отсутствие любых ошибок. Ограничения не скрыты в этом статусе. Общие результаты и команды — VALIDATION.md; внешний SMTP/ФСП и человеческий пилот — not_run.

ID	Реализация	Проверка / факт	Статус и границы
M01	main.py auth, App.tsx Auth	регистрация двух ролей в Chromium, одноразовость/срок/роль в test_api	pass в демо; реальная SMTP-доставка not_run
M02	schemas.ProfileInput, ProfileForm	UI сохранение полей, API профиль	pass, выбран разрешённый ТЗ ручной ввод
M03	pdf.py + единая projection	Cyrillic extraction, запрет второй компании, browser download	pass
M04	bank.py, Assessments	6 категорий, самостоятельный grade, реальные варианты	pass; отрасль MVP только ИТ
M05	Attempt/Evidence/Profile	самодекларация и FSP не присваивают grade; успешная серверная проверка	pass в рубрике MVP
M06	12 семейств, fingerprint, oracle	600 unit-вариантов + 720 вариантов оценки; 2880 независимых проверок в evaluation	реализовано; психометрическая сопоставимость на людях not_run
M07	submit	провал первого/повторного теста и истечение времени	pass, автоматического понижения нет
M08	timestamps, 90 дней / 24 часа	точные границы, история, добровольное изменение	pass
M09	Invitation + decide	принятие/отклонение, чужой кандидат, повтор	pass
M10	Consent + Profile privacy	отзыв/повторная публикация/скрытие стажа, снимки/PDF/чат	pass; удаление аккаунта и политика хранения отложены
M11	fsp_provider.py, FSP UI	привязка/отвязка, без ФСП полный Chromium-сценарий	pass для явно обозначенного mock; реальный реестр not_run
M12	Company/CompanyMember	собственная компания, UI сохранение, разделение двух компаний	pass; один сотрудник на компанию в UI
M13	NeedInput, SearchPage	подтверждённая структура и выдача по категории	pass; без автоматического NLP
M14	GET candidates	spec/grade/verified skill/FSP фильтры, unknown исключён из verified skill	pass
M15	matching.py + evidence UI	0,80 P@5 против 0,475; источник и дата; FSP только внутри категории	pass на синтетике, рыночная релевантность не доказана
M16	Need + Snapshot	новые снимки, возврат, запрет другой компании, актуальная приватность	pass; доказательства исторические, PDF текущий
M17	invite без Vacancy	UI предложение адресному кандидату	pass
M18	InviteInput + DB CHECK	8 негативных комбинаций, положительные целые RUB	pass, ноль запрещён
M19	invitation snapshots	компания/описание/контакт/условия до принятия	pass

ID	Реализация	Проверка / факт	Статус и границы
M20	ContactGrant + projection	полный lifecycle API/поиск/PDF, две компании, raw text	pass, адресное разрешение; скачанное ранее отозвать нельзя
M21	state transitions	sent/viewed/accepted/rejected у сторон, повторы и параллельность	pass
M22	свой backend/БД	сценарий без внешних вакансий и платных моделей	pass
M23	evaluation fixtures/protocol/oracles	явные labels, dev/test, baseline, матрица ошибок и сохранённые провалы	pass как собственная синтетическая процедура; экспертов 0
M24	исходники, lock, compose, migrations	образы собраны; новая БД и повтор seed; Git workflow	pass; код отправлен в GitHub, ветка codex/fsp-mvp; локальная история сохранена
M25	docs/delivery + OpenAPI	схемы 27 операций / 35 моделей, DOCX/PDF, инструкции	реализовано; визуальная проверка документов — release-checks
M26	роли, Argon2, hashed tokens, CSRF, safe views	доступ по чужим ID, cookie, отказ SMTP и rollback, audits	pass в проверенном объёме; производственная аттестация не заявлена
M27	demo + шаблон PPTX	браузерный walkthrough, скриншоты, презентация по обязательным блокам	MVP/демо реализованы; имена/роли по TEAM_TASKS.md; фамилии/контакты и история не предоставлены
R01	looperback стенд + SSH / видео	Compose и Chromium	pass через разрешённую альтернативу видео + локальный запуск; публичного домена нет
R02	один Compose-проект	30 пользователей, 270 чтений, р95 3,04 с, 0 ошибок	pass в ограниченном замере, масштабирование отложено
R03	FSP_INTEGRATION.md	проект OIDC/Keycloak и границы провайдера	проект подготовлен, внешняя интеграция not_run
O01	вакансии/самостоятельный отклик	отсутствуют	не реализовано, необязательное
O02	задания работодателя	отсутствуют	не реализовано, необязательное
O03	Message, Chat	диалог только после принятия и при действующем согласии	pass
O04	адаптивность, явные условия	Chromium 390/1440 px, no overflow	адаптивность pass; ATS и модерация не реализованы
D01	python/data × Junior/Middle/Senior	6 blueprints	pass
D02	React/TS + FastAPI + PostgreSQL	production build + integration + Docker	pass

Ключевые тесты: tests/test_api.py, tests/test_bank.py, tests/test_matching.py, tests/browser_flow.py, tests/load_probe.py. Невыполненное полное ревью стороннего архива остаётся отдельным ограничением, разрешённым к обходу для перехода к реализации последним указанием пользователя.

API 1.0.0

Источник истины — `openapi.json`, выгруженный из работающего FastAPI. Все запросы/JSON-ответы имеют схемы. Даты — ISO 8601, UTC; интерфейс отображает московское время. Неизвестные значения представлены `null/unknown`, а не положительным совпадением. Cookie `HttpOnly` хранит сессию.

Для POST/PUT/PATCH нужен заголовок `X-Requested-With: fsp-web`. Неизвестные поля входа отвергаются: нельзя добавить `verified_grade`, `seed`, `score`, `company_id` или роль в обход нужного маршрута.

Маршрут	Доступ и результат
GET /api/health	состояние БД, версия, режим
GET /api/catalog	направления, навыки, уровни, версия банка, интервалы
POST /api/auth/register	email, пароль 10...128, роль, <code>processing=true</code> ; 201
POST /api/auth/verify	одноразовый token; успех или 400
POST /api/auth/login	подтверждённый email/пароль; устанавливает cookie
POST /api/auth/logout	аннулирует сессию и cookie
GET /api/auth/me	своя роль, ID, email, признак демо
POST /api/auth/demo	только DEMO_MODE; candidate/employer/other_employer
GET/PUT /api/me/profile	только кандидат, собственный профиль и согласия
POST /api/me/fsp	пример demo-winner/demo-participant или unlink
GET/POST /api/me/attempts	история / старт {grade}; сервер выбирает вариант
POST /api/me/attempts/{id}/submit	только владелец; {answers:{"1":"42",...}}; повтор неизменен
GET/PUT /api/me/company	только своя компания
GET /api/candidates	только работодатель; specialization, grade, skill, fsp_only
GET /api/candidates/{id}	опубликованная проекция с текущими правами
GET /api/profiles/{id}/pdf	собственный либо доступный работодателю PDF
POST /api/searches	подтверждённая NeedInput; создаёт потребность и снимок
GET /api/searches	до 50 последних снимков своей компании
GET /api/searches/{id}	только свой снимок; повторная проверка приватности
POST /api/invitations	положительная вилка RUB и UUID request_id; 201
GET /api/invitations	только свои приглашения / приглашения своей компании
PATCH /api/invitations/{id}	только кандидат-адресат; viewed/accepted/rejected
GET/POST /api/invitations/{id}/messages	участники после принятия и при действующем согласии

Пример структурированной потребности:

```
{ "title": "Разработчик API", "description": "Разработка серверных API команды",  
  "specialization": "python", "grades": ["Junior", "Middle"],  
  "required_skills": ["python"], "desired_skills": ["testing"],
```

```
"min_years":null,"fsp_only":false,"confirmed":true}
```

Пример предложения (ID берутся из своих разрешённых ответов, request_id создаётся один раз на попытку отправки и повторяется при сетевом повторе):

```
{ "candidate_id": "00000000-0000-4000-8000-000000000001",  
  "request_id": "00000000-0000-4000-8000-000000000002",  
  "description": "Приглашаем развивать API нашей команды",  
  "salary_min": 120000, "salary_max": 180000, "currency": "RUB" }
```

Ошибки: { "detail": "Понятное сообщение", "fields": [...] }. fields есть при ошибке валидации, содержит только имя/тип ошибки, не исходное значение (пароли и токены в ошибку не возвращаются). 401 — вход/срок сессии; 403 — роль/согласие/CSRF; 404 — недоступная или чужая запись; 409 — конфликт, повтор ключа с другим payload, кулдаун, истечение теста; 422 — поля; 429 — частота; 503 — недоступная доставка почты/интеграция. 404 не обещает, что записи нет физически: может отсутствовать право доступа.

contacts=null до адресного разрешения. Имя и произвольные поля не возвращаются. После принятия выдача, PDF и карточка приглашения используют одно право компании. У другой компании contacts остаётся null.

OpenAPI содержит модели ошибок, cookie security scheme, обязательный заголовок изменения данных и application/pdf. /api/docs — локальная страница со спецификацией; внешние Swagger CDN в runtime не используются.

ФСП ID и будущий Keycloak

В текущем MVP `fsp_provider.py` отделяет интерфейс `Provider` от `DemoProvider`. Пользователь выбирает вымышленный пример победителя или участника. Ответы помечены `provider=demo_fsp`, `verification=simulated`. Это не подтверждение реестром и не реальный FSP ID. Идентификатор примера дополнен локальным ID пользователя, поэтому не подменяет глобальную внешнюю идентичность.

`DEMO_MODE=false` запрещает демонстрационную привязку. Без привязки работают профиль, тест, подбор, приглашение, раскрытие контактов и диалог. Отвязка удаляет текущий спортивный бонус, но не меняет грейд и историю тестов. Историческая подборка сохраняет помеченное доказательство на свою дату.

План реальной интеграции, не исполненная интеграция:

1. Получить у ФСП адрес Keycloak issuer, регистрацию OIDC-клиента, разрешённые redirect URI, API достижений, scopes и тестовые учётные записи. 2. Вход через Authorization Code + PKCE. Проверять state/nonce, issuer, audience, срок и подпись ID token по доверенному JWKS. Не принимать переданный браузером ID/статус победителя как доказательство. 3. Привязка внешнего (issuer, subject) к локальному User только из действующей сессии и после подтверждённого OIDC-перехода. Не объединять аккаунты автоматически по неподтверждённому email. Уникальность пары — в БД. 4. Сервер запрашивает достижения по минимально необходимому scope; хранит provider, внешний ID события, результат, дату, время проверки и источник. Секреты и access/refresh tokens не уходят в браузер и логи. 5. Реализовать отзыв связи, истечение/обновление свидетельства, повторную синхронизацию и понятное состояние недоступности внешнего API. 6. Дополнить контракт провайдера, схемы верификации, миграцию и тесты: чужой subject, повторный callback, неверные issuer/audience/nonce, истёкший токен, подмена достижения, частичная недоступность.

Производственные OIDC-потoki, доступ к реестру и проверка этих интеграционных сценариев не запускались: конфигурация и внешние учётные данные не предоставлены. Достижения останутся ограниченным сигналом внутри категории; присвоение профессионального грейда продолжит зависеть от отдельной процедуры проверки.

Запуск и сопровождение

Инструкции установки — в корневом README. Runtime: Python 3.12, FastAPI 0.142.3, Starlette 1.3.1, SQLAlchemy 2.0.41, Psycopg 3.2.9, Alembic 1.16.1, Uvicorn 0.34.3, Argon2-cffi 23.1.0, ReportLab 4.4.1; React 19.1.0, TypeScript 5.8.3, Vite 6.4.4, Lucide 0.511.0; PostgreSQL 16, Nginx 1.28. Точные транзитивные версии/хеши Python и версии npm закреплены в lock-файлах. Базовые Python/Node/Nginx image tags могут получать исправления upstream; PostgreSQL образ закреплён digest.

Сборка выполняется без параллельного запуска двух frontend-сборок на маленьком сервере. Для ограничения памяти используются точечные импорты иконок и отключён рекурсивный Rollup tree-shaking; esbuild сохраняет минификацию. Это осознанная настройка, проверенная реальной сборкой.

Данные и резервирование

Миграции выполняются перед стартом API. Additive seed работает только при DEMO_MODE=true, не перезаписывает существующие аккаунты/попытки. Автоматической очистки демонстрации нет. После прохождения общего аккаунта используйте новую синтетическую регистрацию, а не удаление тома и не обход серверного календаря.

Резервная копия только своего проекта:

```
mkdir -p local-backups
docker compose exec -T db pg_dump -U fsp -d fsp -Fc > local-backups/fsp.dump
```

Копия содержит персональные данные, если оператор допустил их ввод; хранить в закрытом месте вне git. Восстановление сначала проверять в отдельной пустой БД этого же проекта. Не применять глобальные Docker prune/down -v для обновления приложения. Для остановки достаточно docker compose stop.

Переход из демонстрации

Для реальной доставки нужны SMTP_HOST/PORT/FROM, при необходимости USER/PASSWORD, SMTP_TLS=true и DEMO_MODE=false. При отсутствии доставки регистрация откатывается и возвращает 503, токен не показывается. При рабочем SMTP в ответе нет демонстрационной ссылки. Реальная отправка в этой сдache не запускалась.

Перед публичным использованием оператор должен настроить собственный HTTPS reverse proxy, PUBLIC_URL и COOKIE_SECURE=true. Текущий стенд — loopback HTTP для локального показа. Приложение не заявлено как готовое производственное решение для персональных данных: нужны правила хранения, удаления/запросов субъекта, юридические документы оператора и оценка размещения. Наличие checkbox и журналов согласия не является заявлением о полной юридической сертификации. Реальные CV для разработки не использовались.

Пароли, cookie, verification-токены, содержимое CV и .env не публикуются в репозитории. Access log Uvicorn выключен. Email token передаётся во фрагменте URL и удаляется frontend из адресной строки перед подтверждением.

Ограничения эксплуатации

- Rate limit локален одному процессу; за Nginx в текущей конфигурации он делится клиентами одного проху. Для публичного потока нужен limiter на доверенном внешнем адресе / общий backend и отдельная политика демовхода.

- Банк/подбор используют небольшую выборку без пагинации и оптимизации всех N+1 запросов. Замер относится к десяткам одновременных запросов и текущему демонабору; не к миллионам резюме.

- Нет восстановления пароля, повторной отправки письма, удаления аккаунта, UI управления сотрудниками, модерации работодателей, уведомлений по почте.
- Сессии/токены с истёкшим сроком отвергаются, но их фоновая очистка отложена.
- Нет реальной ФСП/OIDC-интеграции, прокторинга, импортов файлов и семантического поиска. ФСП-адаптер и план замены описаны отдельно.
- Общий демовход предназначен только для вымышленных данных; его нельзя оставлять включённым в среде реальных кандидатов.

Материалы исходного архива сохранены. Полное трёхпроходное чтение всех его файлов не завершено. Автоматические хеши/сканы и содержательное чтение разделены в docs/review; разрешение перейти к коду не меняет эти статусы.